Understanding Our Milling Data

An Explainer for the Whole Team

MDP PRISM Project | University of Michigan

1. What Is This Project About?

We are building a machine learning (ML) system that can predict whether a milling machine will experience a problem called chatter — a violent, noisy vibration that damages both the cutting tool and the workpiece. Think of it like a steering wheel shaking at high speed on a bad road: uncomfortable, destructive, and avoidable if you know the right speed.

We are using a publicly available database called the Digital Machining Database, hosted on Kaggle (a free online platform for data science). It contains 66 gigabytes of physics-based simulated milling data across 60 different tool and material setups.

Key goal: Given a spindle speed and cutting depth, predict whether the machine will chatter — and explain WHY the model made that prediction.

2. What Is Milling? (The Physics in Plain English)

Milling is a manufacturing process where a spinning cutting tool (called an endmill) removes material from a workpiece — like a rotating drill carving a channel in metal. The machinist controls two main things:

- How fast the tool spins — the spindle speed, measured in RPM.
- How deep the tool cuts — the axial depth of cut, measured in millimetres.

2.1 The Stability Boundary

There is a threshold beyond which the machine starts chattering. That threshold is the stability boundary. Below it = safe. Above it = chatter. The boundary is not a straight line — it forms a wavy curve of mountain peaks called stability lobes when plotted against spindle speed. At lobe peaks the machine is very stable; in the valleys even a shallow cut causes chatter.

Simple rule: below the boundary = safe cut. Above the boundary = chatter. Our ML model learns to predict which side any given cut falls on.

2.2 Why Chatter Happens

When the tool removes material it leaves tiny waves on the surface. On the next tooth pass the tool cuts over those waves again. If the timing is wrong, the waves amplify each other — like a microphone too close to a speaker — and the tool shakes violently. This is chatter. The physics governing it involves a time delay (one tooth-pass period) which makes it a delay-differential equation (DDE) — harder than a standard equation because the current behaviour depends on what happened one tooth-pass ago.

Term	Plain English Meaning	Example Value
Spindle speed (Ω)	How fast the tool spins	2,000 – 12,000 RPM
Axial depth (b)	How deep the tool cuts	1 mm – 15 mm
Stability boundary (blim)	Maximum safe depth at a given speed	Varies — 1.47 mm at 2,000 RPM
Tooth pass frequency (TPF)	Impacts per second (speed × teeth / 60)	133 Hz at 2,000 RPM, 4 teeth
Natural frequency (fn)	Frequency at which the tool vibrates on its own	500 Hz in Dataset 1
Delay (τ)	Time between one tooth and the next	7.5 ms at 2,000 RPM, 4 teeth
Chatter frequency	Frequency of the chatter vibration	~502 Hz in Dataset 1 (just above fn)

3. What Is in the Database?

What	Count
Total datasets	60
Total simulation files	18,669
Stable (safe) simulations	9,772 (52.3%)
Unstable (chatter) simulations	8,897 (47.7%)
Stability boundary files	60 — one per dataset
Total size on disk	66 GB

The data is nearly 50/50 stable vs unstable — ideal for ML. No special corrections needed.

3.1 File Types

- Each simulation is saved as both a .h5 file (Python-friendly HDF5) and a .mat file (MATLAB). They are identical. We only use the .h5 files.
- Each dataset folder contains: many time-series files (one per speed+depth simulation) and one stability boundary file.

4. What Is the Omega-B Grid?

Every simulation file represents one specific combination of spindle speed (omega) and axial depth (b). The full set of all combinations tested is the omega-b grid — a table where each cell is one .h5 file. The grid was deliberately designed to always straddle the stability boundary: b=1mm is always stable, and the two deepest values at each speed are always unstable.

The grid is simply the map of all speed + depth combinations simulated. Each cell = one file.

5. What Is Inside Each Simulation File?

Each .h5 file contains 17 channels recorded at 22,560 time steps over 1.8 seconds (sampling rate ~12,500 Hz):

#	Variable	Plain English	Unit
1	omega	Spindle speed (single number)	RPM
2	b	Axial depth (single number)	metres
3	time	Clock timestamps	seconds
4–5	xtpos / ytpos	How far the tool moved left-right / front-back	metres
6–7	xwpos / ywpos	How far the workpiece moved left-right / front-back	metres
8–9	xtvel / ytvel	How fast the tool is moving in each direction	m/s
10–11	xwvel / ywvel	How fast the workpiece is moving in each direction	m/s
12–13	xtacc / ytacc	How quickly the tool is accelerating	m/s²
14–15	xwacc / ywacc	How quickly the workpiece is accelerating	m/s²
16	Forcex	Cutting force left-right	Newtons
17	Forcey	Cutting force front-back	Newtons

6. How We Label Each File as Stable or Unstable

Simulation files have no built-in label. We assign labels by comparing each file's axial depth to the stability boundary at that spindle speed:

- b < blim → Label 0 (STABLE)
- b > blim → Label 1 (UNSTABLE / chatter)

File	Speed	Depth (b)	Boundary (blim)	b > blim?	Label
time1_1.h5	2,000 RPM	1.0 mm	1.473 mm	No	0 — STABLE
time1_2.h5	2,000 RPM	2.0 mm	1.473 mm	Yes	1 — UNSTABLE
time1_3.h5	2,000 RPM	3.0 mm	1.473 mm	Yes	1 — UNSTABLE
time1_8.h5	2,500 RPM	5.0 mm	4.863 mm	Yes	1 — UNSTABLE

7. What Changes Across the 60 Datasets?

Parameter	Plain English	Range
Natural frequency (fn)	How fast the tool vibrates on its own	500 – 1,500 Hz
Cutting force coefficient (ktc)	How hard the material is to cut	700 – 2,000 MPa
Number of teeth (m)	Cutting edges on the endmill	2, 3, 4, or 5
Process damping (Cn)	Extra damping at slow speeds (some datasets only)	0 – 400,000 N/m
Cut direction	Up milling vs down (climb) milling	1 or 2
Tool diameter	Physical size of the endmill	12.7 mm or 19.05 mm
Spindle speed range	Min and max speed simulated	1,000 – 30,000 RPM

Because fn changes across datasets, all frequency-based features must be expressed as ratios (e.g. chatter frequency / fn) rather than absolute Hz values. This ensures the ML model generalises across all 60 datasets.

8. What Do the Signals Actually Look Like?

We produced three types of visualisation from the actual simulation data. Each one reveals a different aspect of how stable and unstable cuts differ. All plots compare the same two conditions: omega = 2,000 RPM, stable cut at b = 1 mm (blue) vs unstable cut at b = 2 mm (red).

8.1 Time-Domain Plot — Watching the Tool Vibrate Over Time

The most direct view: how far the tool moved (in micrometres, μm) and how hard it pushed (in Newtons) at each moment during the 1.8-second simulation. Think of it like a hospital heart-rate monitor — you are watching the tool's 'heartbeat' over time.

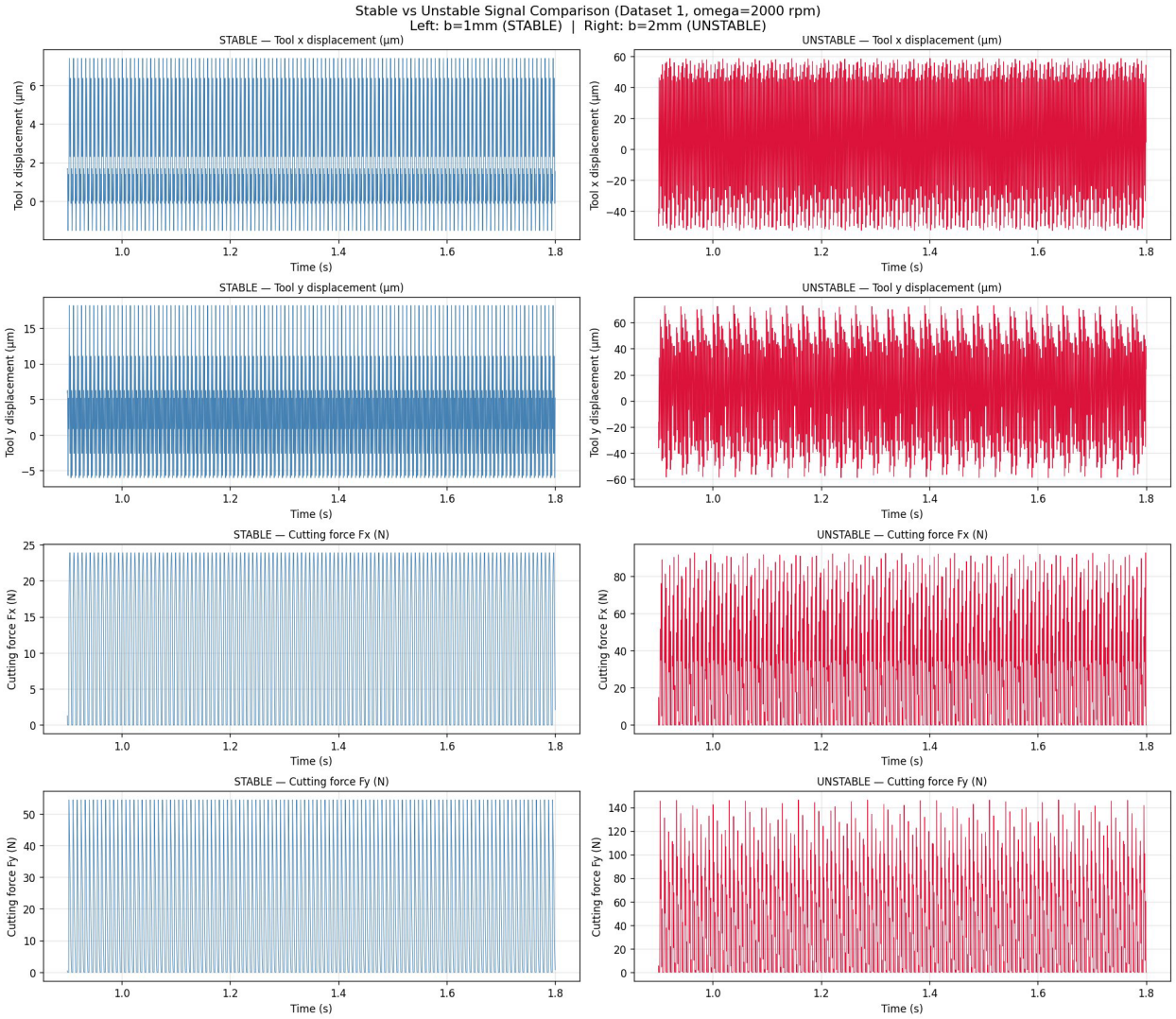


Figure 1: Time-domain signals for stable (blue, b=1mm) vs unstable (red, b=2mm) at omega=2,000 RPM. Rows: tool x displacement, tool y displacement, cutting force Fx, cutting force Fy.

🔍 What you are seeing in Figure 1

- Blue (STABLE) — left column: Every signal is a clean, tight band of repeating peaks. The tool vibrates with each tooth pass (133 times per second at 2,000 RPM) but the vibration never grows — it stays contained within a consistent range. Tool x-displacement stays between roughly -5 µm and +7 µm. Forces stay under 25 N. The pattern is perfectly predictable.
- Red (UNSTABLE) — right column: The signals are dramatically wider. Tool x-displacement swings from -40 µm to +60 µm — roughly 8× the amplitude of the stable case. The force Fx hits 90 N (vs 25 N stable) and Fy hits 140 N (vs 55 N stable). The pattern looks more chaotic because two frequencies are fighting each other: the tooth-pass frequency (133 Hz) and the chatter frequency (~502 Hz).
- The key difference is NOT just that the unstable signal is bigger. It is that a completely different frequency has taken over — chatter. The irregular, noisy appearance of the red signals is the visual signature of those two competing frequencies.
- Why the stable signal has two nested bands: the inner band is the average position of the tool; the outer band is the vibration around that average. In the unstable case, the outer band is so large it swallows the inner structure entirely.

8.2 FFT Plot — The Frequency Fingerprint

The Fast Fourier Transform (FFT) converts the raw time signal into a frequency fingerprint — it answers the question: which frequencies are present and how strong are they? Think of it like a music equaliser bar chart: instead of seeing the sound change over time, you see exactly which notes are playing and how loud each one is.

What you expect to see in a stable cut:

With 4 teeth at 2,000 RPM, the tool hits the workpiece 133 times per second. The FFT should show peaks at 133 Hz (the tooth-pass frequency) and its harmonics — 266, 400, 533 Hz, and so on. These are the 'expected notes' of a healthy cut. Nothing else should appear.

What appears when chatter starts:

A brand new peak appears near the natural frequency of the tool (500 Hz in Dataset 1). This peak is not a multiple of 133 Hz — it sits at 502 Hz, and 502 / 133 = 3.77, which is not a whole number. Mathematical sidebands appear around it at 502 ± 133 = 369 Hz and 635 Hz. This non-integer ratio is the clearest mathematical signature of chatter.

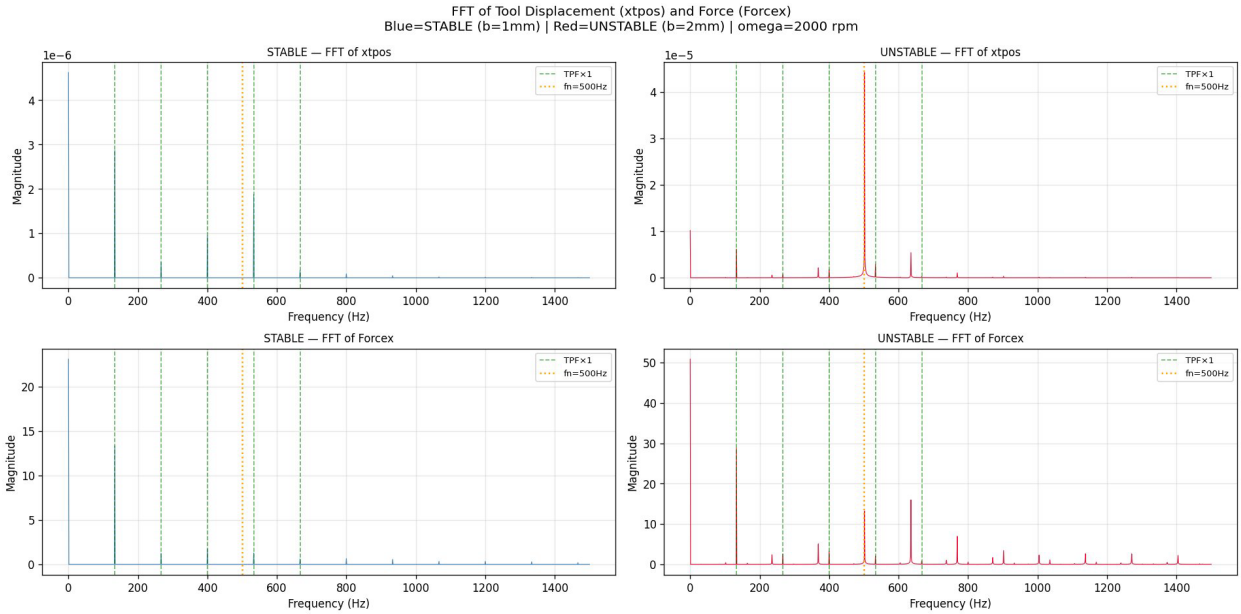


Figure 2: FFT of tool displacement (top row) and cutting force (bottom row) for stable (left, blue) and unstable (right, red). Green dashed lines mark tooth-pass harmonics. Orange dotted line marks the natural frequency (500 Hz).

🔍 What you are seeing in Figure 2

- Top-left (STABLE xtpos FFT): Three clean spikes at exactly the tooth-pass frequency (133 Hz) and its harmonics (266 Hz, 400 Hz, 533 Hz). The orange line at 500 Hz has almost nothing near it — the tool is not vibrating at its natural frequency. Everything is orderly.
- Top-right (UNSTABLE xtpos FFT): A massive spike has appeared just above 500 Hz (at ~502 Hz). This is the chatter frequency. Notice the scale on the y-axis has changed from 1e-6 to 1e-5 — the unstable peak is roughly 10× taller than anything in the stable plot. The tooth-pass spikes are still there but look tiny by comparison.
- Bottom-left (STABLE Forcex FFT): The force follows the same pattern — dominant peak at 0 Hz (average force) and at 133 Hz. Very clean, very predictable.
- Bottom-right (UNSTABLE Forcex FFT): Completely different character. The chatter frequency (502 Hz) and its sidebands (635 Hz, 768 Hz) are now as large as the tooth-pass peaks. The machine is fighting two different rhythms at once. The force is much more variable and unpredictable.
- The non-integer ratio (502/133 = 3.77) is our strongest ML feature. In a stable cut this ratio is always a whole number (1, 2, 3...). In a chatter cut it is always non-integer. No other feature separates the classes as cleanly.

```
STABLE cut — top frequencies in xtpos:
  133 Hz  →  tooth-pass frequency        (expected)
  400 Hz  →  3rd harmonic of 133 Hz      (expected)
  533 Hz  →  4th harmonic of 133 Hz      (expected)
  No peaks near 500 Hz
```

```
UNSTABLE cut — top frequencies in xtpos:
  502 Hz  →  CHATTER FREQUENCY  (new! 502 / 133 = 3.77, not a whole number)
  133 Hz  →  tooth-pass still present but weaker
  635 Hz  →  sideband: 502 + 133 = 635  (confirms chatter)
  768 Hz  →  sideband: 502 + 266 = 768  (confirms chatter)
```

8.3 Phase Portrait — The Shape of Motion

A phase portrait plots displacement against velocity simultaneously at every moment. Instead of asking 'where was the tool at time t?', it asks 'what shape does the tool trace out in position-velocity space?'. Think of drawing a picture of the tool's motion rather than watching it happen in real time.

If a system vibrates in a perfectly regular repeating pattern, its phase portrait is a closed loop — the tool traces the exact same path over and over. More complex or changing motion produces thicker, filled-in shapes.

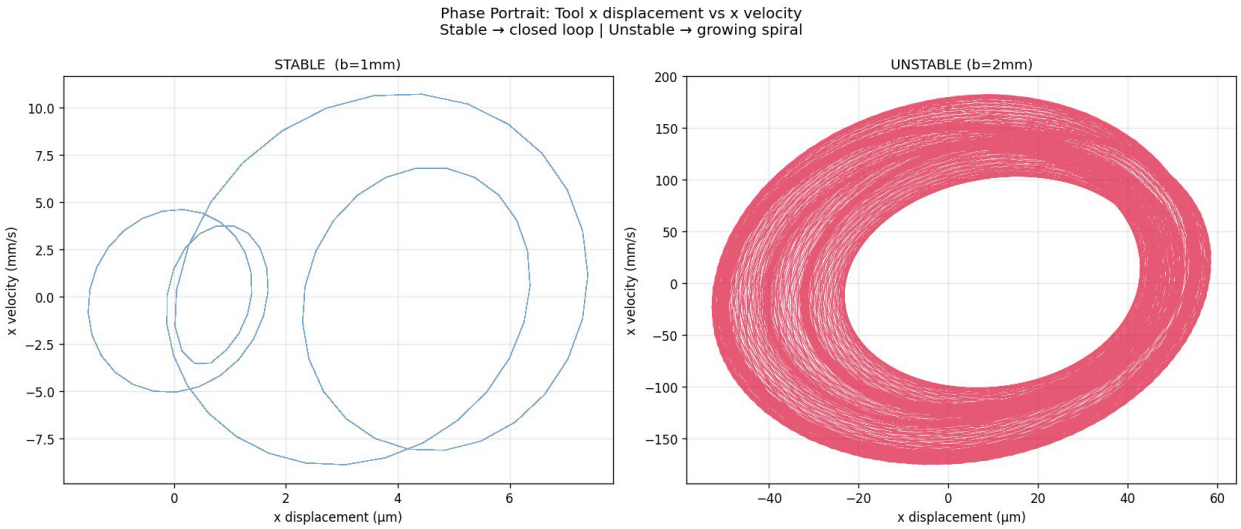


Figure 3: Phase portrait (tool x displacement vs x velocity) for stable (left, blue) and unstable (right, red). Stable: sparse overlapping loops. Unstable: densely filled band.

🔍 What you are seeing in Figure 3

- Left (STABLE, blue): A small number of open, non-overlapping loops. The tool traces a few distinct elliptical paths — one for each tooth. The loops do not all land on top of each other because the tool's average position drifts slightly with each revolution (the multiple loops represent different tooth positions). Crucially, the loops are THIN and well-defined, meaning the motion is highly predictable and repeatable.

- Right (UNSTABLE, red): A densely filled elliptical band. The tool is tracing out the same general elliptical shape many times, but the chatter frequency (502 Hz) causes the path to shift slightly each pass — creating the thick filled band. The displacement range has grown from about 7 µm wide to 60 µm wide. The velocity range has grown from about 15 mm/s to over 300 mm/s.
- The key insight: both portraits are elliptical, meaning both are periodic. But the stable one has a simple, single-frequency periodicity (tooth-pass). The unstable one has quasi-periodicity — two incommensurate frequencies (133 Hz and 502 Hz) beating against each other, which is why the band is thick rather than a thin line.
- Why this matters for SINDy: SINDy discovers the governing equations by finding a sparse mathematical law that produces these shapes. The stable phase portrait can be described by a simple damped oscillator equation. The unstable portrait requires the delay-differential equation — which is exactly what SINDy-DDE is designed to identify.

9. Libraries and Tools We Are Using

Library	What It Does	Where We Use It
h5py	Reads the .h5 simulation files from disk	Loading every time-series and boundary file
numpy (np)	Fast numerical calculations — averages, FFT, square roots	Signal statistics, FFT, labelling
pandas (pd)	Organises data into labelled tables for inspection and analysis	Class balance tables, summaries, grids
matplotlib.pyplot (plt)	Draws and saves all charts and figures	Figures 1, 2 and 3 above
scipy.signal	Advanced signal processing — filtering, spectral analysis	FFT and frequency analysis
os	Navigates the file system — lists folders, builds file paths	Finding all .h5 files across 60 folders

10. What Each Section of Our Code Does

Part 1 — Inventory All Datasets

Scans all 60 dataset folders and counts files. Key finding: Dataset 29 has 1,071 files; Dataset 1 has 102.

Part 2 — Read the Descriptions Spreadsheet

Reads the physical parameters for all 60 datasets — natural frequencies, stiffness, cutting coefficients, number of teeth. Essential for building normalised features.

Part 3 — Label Every File

Assigns Label 0 or 1 to every file by comparing b against the stability boundary. No labels = no ML training.

Part 4 — Check Class Balance

9,772 stable (52%) vs 8,897 unstable (48%) overall. Nearly perfect balance. Individual datasets vary from 23% to 73% unstable.

Part 5 — Analyse Signal Characteristics

Reads time-series from sample files and computes statistics: RMS amplitude, peak-to-peak, FFT fingerprint, phase portrait. Tells us which features will work for the ML classifier.

11. Key Findings

Summary of the most important things learned from exploring the data.

Finding 1 — Class Balance Is Excellent

47.7% unstable across 18,669 files. No class imbalance correction needed.

Finding 2 — The FFT Chatter Signature Is Unmistakable

Chatter always produces a peak near fn (502 Hz) that is not a harmonic of the tooth-pass frequency. The ratio 502/133 = 3.77 is non-integer — this is the strongest single ML feature.

Finding 3 — Simple Amplitude Statistics Are Weak Alone

RMS displacement is only 1.2× higher in unstable cuts. Peak-to-peak force is 1.4× higher. FFT frequency features are far more discriminative.

Finding 4 — Features Must Be Normalised Across Datasets

fn varies (500 to 1,500 Hz across datasets). All features must use ratios relative to fn, not absolute Hz values.

Finding 5 — 240 Delay Periods Per File Is Excellent for SINDy

Each simulation contains 240 full tooth-pass delay cycles. SINDy-DDE needs many complete cycles to identify the governing equation reliably.

12. Model Development Plan

This section describes each model we will build, why we chose it, how it connects to the others, and what we expect it to produce.

12.1 Feature Extraction (Foundation Step)

Before any model can be trained, we need to convert all 18,669 raw simulation files into a single flat table where every row is one simulation and every column is a computed number. This is called feature extraction. Without it, the ML models have nothing to learn from.

We extract three categories of features from each file:

- Time-domain features: RMS amplitude, peak-to-peak swing, and kurtosis of tool displacement and cutting force signals. These capture how big and how irregular the vibrations are.
- Frequency-domain features: dominant frequency, frequency ratio (dominant frequency divided by tooth-pass frequency), proximity to the natural frequency, and the energy ratio between the chatter frequency band and the tooth-pass band. These are the strongest predictors.
- Physics parameters: spindle speed, axial depth, natural frequency, cutting force coefficient, number of teeth, cut direction, and process damping flag. These come from the descriptions spreadsheet and anchor the features in real physics.

> Critical rule: all frequency features must be expressed as ratios relative to each dataset's natural frequency. A raw frequency of 502 Hz only means something in Dataset 1. The ratio 502/500 = 1.004 means something across all 60 datasets.

The output is a CSV file of approximately 18,669 rows and 30 columns — roughly 10 MB. All subsequent modeling reads only this CSV, never the raw .h5 files again. This is important for speed and reproducibility.

12.2 Baseline Classifier — XGBoost

XGBoost (Extreme Gradient Boosting) is a tree-based machine learning model that works exceptionally well with tabular data. It builds many small decision trees sequentially, each one correcting the errors of the previous one.

We use XGBoost as the primary classifier because: it handles mixed-type features (continuous numbers like RMS and categorical values like cut direction) without any preprocessing, it is robust to feature scale differences, it naturally produces feature importance scores that feed directly into SHAP, and it trains in seconds rather than hours.

Aspect	Detail
Input	Feature table: 18,669 rows, ~30 columns
Output	Probability of chatter for each simulation (0 to 1)
Validation	Leave-dataset-out: train on 54 datasets, test on 6 unseen datasets
Primary metric	AUC-ROC (measures how well the model separates stable from unstable)
Secondary metric	F1-score (balance of precision and recall)
Hyperparameters to tune	Tree depth (3–7), number of trees (100–500), learning rate (0.01–0.1)
Expected performance	90%+ AUC on same-dataset test, 75–85% on leave-dataset-out

The most important validation choice is leave-dataset-out. If we test on files from the same datasets we trained on, the model memorises the specific physics of those datasets and gives falsely optimistic results. Testing on entirely unseen datasets is the honest measure of whether the model has learned general physics rather than dataset-specific patterns.

12.3 Explainability Layer — SHAP, LIME, PDP, DiCE

A model that predicts correctly but cannot explain its reasoning is not useful for engineers or machinists. The explainability layer answers four different questions:

Method	Question It Answers	Scale	What We Expect to Find
SHAP	Which features matter most overall across all 18,669 predictions?	Global	Frequency ratio and fn proximity ranked 1st and 2nd
LIME	Why did the model predict chatter for THIS specific cut?	Local (one file)	Chatter frequency peak identified as the driver for boundary cases
PDP / ICE	How does chatter probability change as spindle speed increases?	Marginal	Sharp drop in chatter probability at lobe peaks (7,500 RPM)

Method	Question It Answers	Scale	What We Expect to Find
DiCE	What is the smallest change to inputs that would make this cut stable?	Local (actionable)	'Change omega from 3,000 to 7,500 RPM at the same depth'

DiCE (Diverse Counterfactual Explanations) is the most practically useful of the four. A machinist running a job at an unstable condition does not need to know that the frequency ratio was 3.77 — they need to know exactly what to change. DiCE answers that directly. We will constrain DiCE to recommend only spindle speeds and depths that exist in the real dataset, so the recommendations are grounded in simulated and verifiable conditions.

12.4 SINDy-DDE — Equation Discovery

SINDy (Sparse Identification of Nonlinear Dynamics) is an algorithm that discovers the governing mathematical equations of a physical system directly from measurement data. It finds the simplest possible equation — the one with the fewest terms — that explains the data. The result is a human-readable mathematical law, not a black box.

The standard SINDy algorithm handles ordinary differential equations (ODEs). We use the extended version called SINDy-DDE, which handles delay-differential equations — essential because milling chatter depends on what the tool was doing one tooth-pass ago, not just right now.

What SINDy-DDE does in plain terms: it reads the time-series of tool displacement, computes the velocities and accelerations, and finds the smallest set of mathematical terms that correctly predicts how the tool moves from one moment to the next, including the delayed term. For a clean stable cut it should recover a simple damped oscillator. For a chatter cut it should recover the regenerative delay term that makes the vibration grow.

We will run SINDy-DDE on 5–10 representative files per dataset — a mix of clearly stable, clearly unstable, and boundary cases. We do not run it on all 18,669 files because equation discovery is computationally expensive (minutes to hours per file) and the goal is to understand the physics, not to label every file.

12.5 PINN — Physics-Informed Neural Network

A PINN is a neural network that is trained to obey a physical law at the same time as fitting the data. Standard neural networks only minimize the gap between predictions and measurements. A PINN additionally minimizes how much the predictions violate the governing equations — discovered by SINDy-DDE in the previous step.

The practical value is generalization. A pure data-driven neural network trained on the simulated grid can only predict signals at the exact (omega, b) combinations it saw during training. A PINN constrained by the physics can predict signals at any (omega, b) combination, including ones never simulated, because it has learned the underlying physical law.

For our project the realistic scope is: train the PINN on Dataset 1 only, hold out 20% of the (omega, b) grid, and show that the PINN predicts the held-out signals more accurately than an unconstrained neural network. That comparison — physics-constrained versus unconstrained — is the central scientific result of the PINN component.

Component	Role in Pipeline
SINDy-DDE output	Provides the delay-differential equation terms used as physics loss
PINN inputs	Spindle speed omega, axial depth b, time t
PINN outputs	Predicted tool displacement xtpos, cutting force Forcex at time t
Physics loss	How much the PINN prediction violates the SINDy-discovered DDE
Data loss	How much the PINN prediction differs from the actual simulation signal
Total loss	Weighted sum of physics loss and data loss — both must decrease during training

13. Why SINDy-DDE and Not Regular SINDy?

This is one of the most important technical decisions in the project and deserves a full explanation. The short answer is: regular SINDy will give you wrong equations for milling chatter, not just incomplete ones.

13.1 What Regular SINDy Assumes

Standard SINDy assumes the system follows an ordinary differential equation (ODE). An ODE says: the future behaviour of the system depends only on its current state. Mathematically:

Regular SINDy assumption (ODE):

- The tool velocity right now depends only on: the tool position right now, the tool velocity right now, and the cutting force right now.
- Nothing in the past matters. Only the present moment.
- This is written as: x_double_dot(t) = f(x(t), x_dot(t))

13.2 What Milling Chatter Actually Is

Milling chatter is fundamentally caused by the regenerative effect. When the tool cuts, it leaves a wavy surface. On the next tooth pass, the tool cuts over those waves. The current chip thickness depends on where the tool is right now minus where it was one tooth-pass ago. That 'one tooth-pass ago' is the delay.

🔍 **Milling reality (DDE):**

- The tool acceleration right now depends on: the tool position right now AND the tool position one tooth-pass ago (the delay tau).
- Tau = 60 / (spindle speed x number of teeth)
- At 2,000 RPM with 4 teeth: tau = 60 / (2000 x 4) = 7.5 milliseconds
- This is written as: x_double_dot(t) = f(x(t), x(t - tau))
- The past matters. Specifically, 7.5 milliseconds ago matters.

13.3 What Happens If You Use Regular SINDy Anyway

If you ignore the delay and run standard SINDy on a chatter signal, three things go wrong:

- SINDy tries to fit an ODE to data that is fundamentally governed by a DDE. It cannot find the correct equation because the correct equation does not exist in the ODE family. It will find the best-fitting ODE approximation instead, which will be wrong.
- The recovered equation will fail to predict chatter onset. It might fit the stable signals reasonably well (because stable cuts are closer to simple ODE behaviour) but will give nonsensical predictions for unstable cuts.
- The discovered terms will not match the known physics. The dataset description tells us exactly what the true governing equation looks like. Regular SINDy will not recover it. SINDy-DDE will — and we can verify this directly because we know the ground truth.

The delay tau is not a small correction or an edge case. It IS the mechanism of chatter. An equation without the delay term is not an approximation of the chatter physics — it is a description of a completely different physical system.

13.4 How SINDy-DDE Solves This

SINDy-DDE extends the candidate library of basis functions to include delayed terms. Instead of only considering x(t), x_dot(t), x(t)^2 and so on, it also includes x(t-tau), x_dot(t-tau), and combinations thereof. The sparse regression then selects which terms — delayed or not — best explain the data.

The delay tau itself can be unknown. SINDy-DDE uses a bilevel optimization: for each candidate tau value, run SINDy and measure the reconstruction error. The tau that gives the

lowest error is selected. In our case we already know tau from the spindle speed and tooth count, so we can provide it directly — making the problem easier and faster than the general case.

	Regular SINDy	SINDy-DDE
Equation type	ODE — current state only	DDE — current state + delayed state
Delay term	Not included	Included as candidate basis function
Can recover chatter physics?	No	Yes — directly recovers the regenerative term
Computation time	Fast (seconds)	Slower (minutes to hours per file)
Applicable to stable cuts?	Yes (reasonable approximation)	Yes (exact)
Applicable to unstable cuts?	No (fundamentally wrong model class)	Yes
Validates against known equations?	Partially	Fully
Python library	PySINDy (standard)	PySINDy with delay library extension

14. Monte Carlo Simulation — Where and Why

Monte Carlo simulation is a technique that runs the same calculation hundreds or thousands of times with slightly different random inputs drawn from a probability distribution, then studies the distribution of outputs. The name comes from the Monte Carlo casino — it is essentially organised, purposeful randomness. In our project it can be used in five distinct places, each serving a different purpose.

14.1 Uncertainty on Stability Boundary Labels (Highest Priority)

This is the most important application. Our current labeling logic is a hard binary decision: b = 1.48 mm with blim = 1.473 mm gives Label 1 (unstable). But the stability boundary is computed from modal parameters (natural frequency, stiffness, damping) that in a real factory have measurement uncertainty. A ±5% error in natural frequency shifts the entire boundary curve.

Monte Carlo quantifies this uncertainty. Instead of one boundary, we compute thousands:

- Sample fn, stiffness k, and damping ratio zeta from Gaussian distributions centred on their nominal values (±5% standard deviation)
- Recompute the stability boundary for each sample
- Run N = 1,000 iterations

- For each (omega, b) point, record what fraction of iterations labelled it unstable

The result is a probability of instability for each simulation file rather than a hard 0 or 1. Points deep in the stable region get probability near 0. Points deep in the chatter region get probability near 1. Points right on the boundary get 0.3 to 0.7 — genuinely uncertain. These probabilistic labels are more honest than binary labels and make the ML model better calibrated.

This directly addresses a real-world problem: no machinist knows their tool's modal parameters exactly. A model trained on probabilistic labels is more trustworthy in deployment than one trained on falsely precise binary labels.

14.2 Robustness Testing of DiCE Counterfactuals

DiCE tells a machinist to change spindle speed from 3,000 to 7,500 RPM to avoid chatter. But that recommendation was generated assuming exact modal parameters. In a real factory, tool wear changes stiffness and temperature changes damping over the course of a shift. Will the recommendation still hold?

Monte Carlo stress-tests each DiCE recommendation:

- Take the recommended (omega*, b*) from DiCE
- Sample modal parameters from uncertainty distributions N = 500 times
- For each sample, check whether (omega*, b*) is still stable
- Report the stability rate: '94% of simulations confirm this recommendation is safe'

A recommendation with 95%+ stability rate is confident. One with 60% stability rate is fragile — the machinist should be warned to move further from the boundary. This converts DiCE from a point recommendation into a risk-aware recommendation, which is far more useful for real manufacturing.

14.3 SINDy Noise Robustness Study

The simulation data is clean — no sensor noise. Real milling machines are not clean. Before claiming SINDy-DDE works in practice, we must show how it degrades as noise increases.

The Monte Carlo study adds synthetic Gaussian noise to signals at increasing levels (1%, 5%, 10%, 20% of signal amplitude), runs SINDy-DDE N = 100 times at each noise level, and measures how often it recovers the correct sparse equation terms. The result is a noise tolerance curve — the maximum noise level at which SINDy-DDE remains reliable. This directly answers whether the method is usable on real experimental data.

14.4 SHAP Variance Across Training Splits

SHAP feature importance values change slightly depending on which random train/test split was used. Reporting SHAP from a single run is technically cherry-picking. Monte Carlo over N = 50 random splits gives confidence intervals: 'frequency ratio has mean SHAP importance 0.34 ± 0.02, consistently ranked first across all splits.' This is a much stronger and more defensible scientific claim.

14.5 PINN Training Stability

PINNs are sensitive to random initialisation. The same architecture with two different random seeds can converge to very different solutions. Monte Carlo over N = 50 random seeds shows the distribution of final loss values and prediction accuracy, identifying whether the PINN reliably converges or frequently fails. This is standard practice in PINN research papers.

14.6 Limitations of Monte Carlo in This Project

Monte Carlo is powerful but not free of problems. Being honest about its limitations is as important as knowing where to apply it:

Limitation	Plain English Explanation	Implication for Our Project
Assumed distributions	Monte Carlo requires you to specify how uncertain each parameter is — e.g. fn has ±5% Gaussian uncertainty. If the real uncertainty has a different shape (e.g. asymmetric, fat-tailed), the results are wrong.	We are assuming Gaussian uncertainty because it is standard. Real measurement uncertainty may not be Gaussian. Treat results as indicative rather than exact.
Computational cost	Running 1,000 boundary computations or 500 PINN evaluations is expensive. The PINN Monte Carlo in particular may take hours on Kaggle.	Prioritise the boundary label uncertainty (fast) and DiCE robustness (moderate) first. PINN Monte Carlo only after the model is stable.
Independence assumption	Monte Carlo assumes each parameter can vary independently. In reality, fn and k are correlated — a stiffer tool also has a higher natural frequency.	For a more accurate study, sample from a joint distribution. For the initial version, independent sampling is a reasonable simplification.
Does not replace validation	Monte Carlo quantifies uncertainty given a model. It does not tell you whether the model itself is correct. If the stability boundary algorithm has a systematic error, Monte Carlo will confidently propagate that error.	Always check that the boundary algorithm gives physically sensible results before running Monte Carlo on top of it.

Limitation	Plain English Explanation	Implication for Our Project
Interpretation complexity	Probabilistic labels and confidence intervals are harder to communicate to non-technical stakeholders than simple pass/fail predictions.	Reserve probabilistic outputs for the technical paper. For the machinist-facing DiCE recommendations, translate to simple risk categories: Safe, Borderline, Unsafe.

15. Additional Research Areas to Investigate

Beyond the core pipeline, several adjacent research topics could strengthen the project significantly. These are not required for the initial deliverable but are worth tracking as the project matures.

15.1 Surface Location Error (SLE)

Chatter is not the only problem in milling. Even in stable cuts, the tool can be in the wrong position at the moment it exits the workpiece, leaving a dimensional error on the surface. This is called surface location error. The dataset description mentions SLE as one of the phenomena the database can support. Including SLE prediction as a second output alongside chatter classification would broaden the project's scope and practical impact considerably. The same XGBoost model can be extended to predict SLE magnitude as a regression target.

15.2 Transfer Learning Across Datasets

The 60 datasets represent different physical setups. A model trained on 54 datasets and tested on 6 new ones is doing zero-shot generalization — no data from the test datasets was seen during training. A more powerful approach is few-shot transfer learning: give the model 5–10 files from a new dataset and fine-tune it. This mimics real deployment where a factory can collect a small number of test cuts before relying on the model. This is an active research area in manufacturing ML and a natural extension of the leave-dataset-out validation strategy.

15.3 Process Damping Datasets

Datasets 25–32 include non-zero process damping coefficients (Cn). Process damping is an additional stabilising effect that appears at low spindle speeds — it is why the stability boundary rises steeply at speeds below 3,000 RPM in some datasets. Models trained only on zero-damping datasets may fail on process-damping datasets. A dedicated analysis of how process damping affects both the signal characteristics and the model performance is a clean, self-contained sub-study.

15.4 Variable Tooth Spacing and Runout

Several datasets include non-zero runout (RO up to 5 micrometres) — meaning the teeth are not perfectly symmetric. This introduces a once-per-revolution frequency into the FFT that standard chatter detection algorithms miss. Investigating whether the XGBoost classifier and SINDy-DDE correctly handle runout-affected signals, and whether runout changes the optimal counterfactuals generated by DiCE, is a practically relevant extension.

15.5 Relevant Literature to Read Before Modeling

The following papers are directly relevant to this project and should be read before the modeling phase begins:

Paper / Topic	Why It Matters for This Project
Altintas & Budak (1995) — Analytical prediction of stability lobes	This is the algorithm used to compute the stability boundaries in the database. Understanding it ensures the labeling logic is correct and helps interpret boundary shape differences across datasets.
Brunton et al. (2016) — Discovering governing equations from data (original SINDy paper)	The mathematical foundation of SINDy. Read this before attempting to implement SINDy-DDE.
Kaheman et al. (2020) — SINDy-PI: robust parallel implicit SINDy	The implicit SINDy variant handles cases where the equation cannot be written in explicit form. The milling DDE coupling between tool and workpiece may require this.
Raissi et al. (2019) — Physics-informed neural networks (original PINN paper)	The mathematical foundation of PINNs. Essential before implementing the PINN component.
Kopeczi-Bocz et al. (2024) — Data-driven delay identification with SINDy	Directly applies SINDy to delay identification in machining-relevant systems. The most directly applicable SINDy-DDE paper to our project.
Mothilal et al. (2020) — Explaining machine learning classifiers through diverse counterfactual examples (DiCE)	The original DiCE paper. Read the constraint specification section carefully — we need to constrain recommendations to physically feasible spindle speeds.
Schmitz & Smith (2019) — Machining Dynamics textbook	The definitive reference for the physics underlying the entire database. The time-domain simulation used to generate the data is described in Chapter 5.

15.6 Validation Metrics to Report

Different audiences care about different metrics. Plan to report all of the following:

Metric	What It Measures	Audience
AUC-ROC	How well the model separates stable from unstable across all threshold choices	ML researchers — primary metric
F1-score	Balance between catching all chatter cases and avoiding false alarms	Engineers — they care about both missing chatter and false stops
Precision at high recall	At 95% recall (catching 95% of chatter cases), what fraction of alarms are real?	Machinists — false alarms cost money and productivity
Cross-dataset AUC drop	Difference in AUC between same-dataset and leave-dataset-out validation	Reviewers — shows generalisation honestly
DiCE validity rate	Fraction of counterfactuals that survive Monte Carlo stress test	Practical deployment — only report recommendations with >85% survival rate
SINDy term recovery rate	Fraction of correct equation terms recovered across noise levels	Physics community — validates the equation discovery

16. What Comes Next — Updated Roadmap

Phase	What We Do	Key Output	Timeline
1A	Feature extraction from all 18,669 files into one CSV	features_all_datasets.csv	Week 1–2
1B	XGBoost classifier with leave-dataset-out validation	Baseline AUC, confusion matrix	Week 2–3
1C	SHAP global importance + PDP plots	Feature importance ranking	Week 3
1D	LIME on boundary cases + DiCE counterfactuals	Per-prediction explanations and actionable recommendations	Week 3–4
1E	Monte Carlo on boundary labels (N=1000)	Probabilistic labels, uncertainty map	Week 4
1F	Monte Carlo on DiCE recommendations (N=500)	Risk-rated counterfactuals	Week 4–5
2A	SINDy-DDE on 50 representative files	Recovered governing equations, noise tolerance curve	Week 5–6
2B	Validate recovered equations against known physics	Equation accuracy score per dataset	Week 6

Phase	What We Do	Key Output	Timeline
3A	PINN prototype on Dataset 1	Signal prediction for unseen omega-b combinations	Week 7–8
3B	PINN vs unconstrained NN comparison	Generalisation improvement from physics loss	Week 8–9